Universal Intelligence: Mathematical Foundations for Agentic AI

Research Notes — ETH Zurich ML Lecture Series

Master's Programme in Computer Science / Data Science
Department of Computer Science, ETH Zürich

Spring Semester 2026

Contents

1 Historical Context **2**

2 Kolmogorov Complexity and Solomonoff Induction **2**
2.1 Turing Machines and Notation . . . 2
2.2 Kolmogorov Complexity . . . 3
2.2.1 Connection to the Minimum Description Length Principle . . . 4
2.3 Solomonoff Induction . . . 4
2.3.1 The Convergence Theorem . . . 4

3 The AIXI Agent **5**
3.1 Agent–Environment Framework . . . 6
3.2 The AIXI Equation . . . 6
3.3 Optimality Properties . . . 6
3.4 Why AIXI Is Incomputable . . . 7
3.5 The Universal Intelligence Measure . . . 7
3.6 Limitations and Known Issues . . . 7

4 Goedel Machines **8**
4.1 Architecture . . . 8
4.2 Connection to Löb's Theorem . . . 8
4.3 Properties . . . 8
4.4 Formal Structure . . . 9
4.5 Comparison: AIXI vs. Gödel Machines . . . 9

5 Speed Prior and Computable Approximations **9**
5.1 The Speed Prior . . . 9
5.2 AIXItl: A Computable Approximation . . . 10
5.3 MC-AIXI-CTW . . . 10
5.4 Feature Reinforcement Learning . . . 10

6 Connections to Modern Agentic AI **11**

7 Open Problems **12**

8 Annotated Bibliography **13**

Abstract

These notes develop the mathematical theory of universal artificial intelligence, beginning from Kolmogorov complexity and Solomonoff's theory of inductive inference, and culminating in Hutter's AIXI agent—a formal model of a universally optimal reinforcement-learning agent. We present the key definitions and theorems with proof sketches pitched at an audience with strong mathematical maturity. We also cover Schmidhuber's Gödel Machines and Speed Prior, computable approximations such as AIXI*tl* and MC-AIXI-CTW, and connections to modern large-language-model–based agents. The notes close with open problems and an annotated bibliography of foundational and recent work. Throughout, we emphasise the interplay between computability, optimality, and the practical gap that separates the ideal from the implementable.

1 Historical Context

The quest to formalise "intelligence" in mathematical terms is older than computer science itself. Three intellectual streams converge in the theory of universal intelligence:

Algorithmic information theory. In the 1960s, Ray Solomonoff [1], Andrey Kolmogorov [2], and Gregory Chaitin [3] independently introduced what we now call *Kolmogorov complexity*—the length of the shortest program that produces a given string. Solomonoff went further: he used the idea to construct a universal prior over hypotheses and showed that Bayesian prediction with this prior converges to the truth faster than any other predictor (in a precise sense we make formal in §2.3).

Reinforcement learning meets computability. Marcus Hutter [8] unified Solomonoff induction with the theory of sequential decision-making to define the AIXI agent: an agent that, at every time step, acts to maximise expected future reward where the expectation is taken over all computable environments, weighted by their Kolmogorov complexity. AIXI is provably Pareto-optimal over the class of all computable environments, but is itself incomputable. Shane Legg, Hutter's doctoral student, used the framework to propose a formal *measure* of universal intelligence [10], arguably the first rigorous definition of machine intelligence.

Self-referential agents. Jürgen Schmidhuber's Gödel Machines [9] take a different approach: an agent that can rewrite its own code provided it can *prove* that the rewrite will improve its future performance. The framework is rooted in Gödel's self-referential constructions and Löb's theorem.

Modern relevance. Laurent Orseau (DeepMind) has extended the agent–environment formalism to study safely interruptible agents [16], a question of urgent practical importance now that large language models are deployed as autonomous agents. The gap between AIXI and a real-world LLM agent is vast, but the theoretical ideals provide guiding principles—Occam's razor, Bayesian updating, self-improvement—that continue to shape system design.

2 Kolmogorov Complexity and Solomonoff Induction

2.1 Turing Machines and Notation

Fix a prefix-free[1] universal Turing machine U. We write $U(p) = x$ to mean that U, on input p, halts and outputs x. We denote by $|p|$ the length of the binary string p.

[1] A set $S \subseteq \{0,1\}^*$ is *prefix-free* if no element of S is a proper prefix of another. This ensures the Kraft inequality $\sum_{p \in S} 2^{-|p|} \leq 1$ holds, which is essential for the universal prior to be a semi-measure.

2.2 Kolmogorov Complexity

Definition 2.1 (Kolmogorov complexity)**.** The *prefix Kolmogorov complexity* of a string $x \in \{0,1\}^*$ with respect to a prefix-free universal Turing machine U is

$$K_U(x) \;=\; \min\bigl\{|p| : U(p) = x\bigr\}.$$

The quantity $K_U(x)$ captures the intrinsic information content of x: the length of the shortest program (on U) that prints x and halts. Two immediate observations:

1. $K_U(x) \le |x| + c$ for a constant c depending only on U (the "identity program" that just copies its input).
2. K_U is not computable (by a reduction from the halting problem).

Theorem 2.2 (Invariance Theorem)**.** *For any two prefix-free universal Turing machines U and V, there exists a constant c_{UV} (depending on U and V but not on x) such that for all $x \in \{0,1\}^*$,*

$$\bigl|K_U(x) - K_V(x)\bigr| \;\le\; c_{UV}.$$

Proof sketch. Since V is universal, there is a program s for V that simulates U. Given any U-program p for x, the string $s \cdot p$ is a V-program for x, so $K_V(x) \le K_U(x) + |s|$. By symmetry, $K_U(x) \le K_V(x) + |t|$ for some simulator t. Setting $c_{UV} = \max(|s|, |t|)$ gives the result. □

Remark 2.3. The invariance theorem justifies speaking of "the" Kolmogorov complexity $K(x)$ without specifying the reference machine, up to an additive constant. For asymptotic statements this constant is irrelevant.

Example 2.4 (Kolmogorov complexity of structured strings)**.** Consider three strings of length $n = 10^6$:

1. $x_1 = 0101\cdots01$ (the pattern "01" repeated $n/2$ times). A short program: "print '01' $n/2$ times." So $K(x_1) \le \log_2 n + c$ for a small constant c.
2. $x_2 =$ the first n bits of π in binary. A short program computes π to arbitrary precision (e.g., via the Bailey–Borwein–Plouffe formula), so $K(x_2) \le \log_2 n + c'$ for a moderate constant c' (encoding the algorithm plus the desired precision).
3. $x_3 =$ a string produced by n independent fair coin flips. With probability 1, $K(x_3) \ge n - O(\log n)$ (by a counting argument: there are 2^n strings of length n but fewer than 2^k programs of length $< k$).

The first two strings are "simple" (low Kolmogorov complexity) despite being long; the third is "random" (high complexity). This formalises the intuition that randomness is incompressibility.

Proposition 2.5 (Incompressibility)**.** *For every $n \in \mathbb{N}$, at least $2^n(1 - 2^{-c})$ of the 2^n binary strings of length n satisfy $K(x) \ge n - c$.*

Proof. There are at most $\sum_{k=0}^{n-c-1} 2^k = 2^{n-c} - 1 < 2^{n-c}$ programs of length less than $n - c$. Each produces at most one output string, so at most 2^{n-c} strings have $K(x) < n - c$. □

Remark 2.6 (Conditional Kolmogorov complexity). One can also define $K(x \mid y) = \min\{|p| : U(p, y) = x\}$, the complexity of x given y as auxiliary input. The *chain rule* $K(x, y) = K(x) + K(y \mid x^*) + O(\log K(x, y))$ (where x^* is a shortest program for x) is the Kolmogorov-complexity analogue of $H(X, Y) = H(X) + H(Y \mid X)$ in Shannon theory. This conditional version is essential for defining mutual algorithmic information and plays a role in the analysis of AIXI's convergence.

2.2.1 Connection to the Minimum Description Length Principle

The Minimum Description Length (MDL) principle [5, 12] can be understood as a computable relaxation of Kolmogorov complexity. In MDL, one restricts the class of "programs" to a parametric model class $\mathcal{M}$ and defines the description length of data x under model $M \in \mathcal{M}$ as

$$L(x \mid M) + L(M),$$

where $L(M)$ encodes the model and $L(x \mid M)$ encodes the data given the model. Minimising this two-part code is a computable approximation to minimising Kolmogorov complexity.

Remark 2.7 (Why Kolmogorov complexity matters for agents). An agent that maintains beliefs over environments must somehow penalise complex hypotheses. Kolmogorov complexity provides the theoretically optimal complexity penalty: it assigns every hypothesis a prior probability $\approx 2^{-K(h)}$. Any other effective prior can be shown to assign lower probability to some hypotheses by at most a multiplicative constant, but can assign exponentially *higher* probability to none (this is the content of the universality of the Solomonoff prior; see below).

2.3 Solomonoff Induction

Solomonoff [1] posed the question: given a sequence of observations $x_1 x_2 \cdots x_{t-1}$, what is the best prediction for x_t? His answer: use the *universal prior*.

Definition 2.8 (Universal prior / Solomonoff prior)**.** The *universal a priori probability* (or *Solomonoff prior*) of a string $x \in \{0,1\}^*$ is

$$M(x) \;=\; \sum_{\substack{p\,:\,U(p) \text{ outputs a string} \\ \text{beginning with } x}} 2^{-|p|}.$$

Equivalently, $M(x)$ is the probability that U, fed a random infinite binary sequence on its input tape, outputs a string starting with x.

Remark 2.9. M is a *semi-measure*: $M(\varepsilon) \le 1$ and $M(x) \ge M(x0) + M(x1)$ for all x, but we do not necessarily have equality because some programs may not halt. M is not computable, but it is *lower semi-computable* (enumerable from below).

The fundamental property of M is its *universality*:

Theorem 2.10 (Universality of $\boldsymbol{M}$)**.** *For every computable probability semi-measure μ on $\{0,1\}^*$, there exists a constant $c_\mu > 0$ (depending on μ but not on x) such that for all $x \in \{0,1\}^*$,*

$$M(x) \;\ge\; c_\mu \cdot \mu(x).$$

In particular, $c_\mu = 2^{-K_U(\mu)}$ where $K_U(\mu)$ denotes the length of the shortest U-program that computes μ.

Proof sketch. Let p_μ be a shortest program for μ on U, so $|p_\mu| = K_U(\mu)$. The contribution to $M(x)$ from programs of the form $p_\mu \cdot$ (data) already accounts for $2^{-|p_\mu|} \cdot \mu(x)$ of the sum, since μ is a measure and U is prefix-free. □

2.3.1 The Convergence Theorem

The predictive power of Solomonoff induction is captured by the following remarkable result.

Theorem 2.11 (Solomonoff's convergence theorem)**.** *Let μ be any computable measure on infinite binary sequences. If $x_1 x_2 \cdots$ is drawn from μ, then*

$$\sum_{t=1}^{\infty} D_{\mathrm{KL}}\big(\mu(\cdot \mid x_{<t}) \,\big\|\, M(\cdot \mid x_{<t})\big) \;\le\; K_U(\mu) \cdot \ln 2,$$

where $x_{<t} = x_1 \cdots x_{t-1}$ and the KL divergence is between the conditional next-symbol distributions.

Proof sketch. Define the *total expected KL divergence*

$$D \;=\; \sum_{t=1}^{\infty} \mathbb{E}_{\mu}\big[D_{\mathrm{KL}}(\mu(\cdot|x_{<t}) \| M(\cdot|x_{<t}))\big].$$

By telescoping the log-loss ratio,

$$D \;=\; \mathbb{E}_{\mu}\left[\ln \frac{\mu(x_{1:\infty})}{M(x_{1:\infty})}\right] \;=\; D_{\mathrm{KL}}(\mu \| M).$$

By universality (Theorem 2.10), $M(x) \ge 2^{-K_U(\mu)}\mu(x)$, so $\ln(\mu(x)/M(x)) \le K_U(\mu) \ln 2$ for all x. Taking the expectation gives $D \le K_U(\mu) \ln 2$. Since the KL divergence is non-negative at every t, the sum converges, meaning $D_{\mathrm{KL}}(\mu(\cdot|x_{<t}) \| M(\cdot|x_{<t})) \to 0$ for μ-almost all sequences. □

Remark 2.12. The bound is *independent of the length of the sequence.* After at most $K_U(\mu) \cdot \ln 2/\varepsilon$ time steps, the prediction error exceeds ε at most $K_U(\mu) \cdot \ln 2/\varepsilon$ times. This makes Solomonoff induction *optimal* in the sense that no predictor can converge faster on all computable environments simultaneously: the convergence rate for environment μ is controlled only by $K_U(\mu)$, the complexity of μ itself.

Example 2.13 (Solomonoff induction in action)**.** Suppose an agent observes the binary sequence $0, 1, 0, 1, 0, 1, \ldots$. The Solomonoff prior M assigns high posterior weight to short programs that generate alternating bits, because these programs contribute $2^{-|p|}$ to $M(x)$ for small $|p|$. Programs that generate the observed prefix but then deviate (e.g., producing $0, 1, 0, 1, 0, 0, 0, \ldots$) are also present in the mixture, but their contribution is split across many incompatible continuations.

After t observations, the posterior weight on the "alternating" hypothesis overwhelms the weight on complex hypotheses, and the predictive distribution $M(x_{t+1} \mid x_{<t+1})$ concentrates near the correct next symbol. The convergence theorem guarantees that the total prediction error (in KL divergence) is bounded by $K_U(\mu) \ln 2$, which for the alternating sequence is a small constant (since the generating program is very short).

Remark 2.14 (Relationship to Bayesian model selection)*.* Solomonoff induction can be understood as Bayesian model selection over the class of *all* computable models, with the universal prior serving as the prior over models. In conventional Bayesian statistics, one chooses a parametric family $\{p_\theta : \theta \in \Theta\}$ and a prior $\pi(\theta)$; the marginal likelihood $\int p_\theta(x)\, d\pi(\theta)$ plays the role of $M(x)$. Solomonoff induction is the limiting case where Θ is the set of all programs (an infinite countable set) and $\pi(p) = 2^{-|p|}$ is the coding prior.

The key advantage is *universality*: Solomonoff induction does not require choosing a model class. The key disadvantage is *incomputability*: the universal prior M cannot be computed, only approximated from below.

3 The AIXI Agent

We now move from passive prediction to active decision-making. The AIXI agent, introduced by Hutter [6, 8], embeds Solomonoff induction inside a reinforcement-learning loop.

3.1 Agent–Environment Framework

Consider an agent interacting with an environment over discrete time steps $k = 1, 2, \ldots$. At each step k:

1. The agent selects an action $a_k \in \mathcal{A}$.
2. The environment responds with an observation $o_k \in \mathcal{O}$ and a reward $r_k \in \mathcal{R} \subset [0, 1]$.

The agent's goal is to maximise cumulative reward. We write the history up to time t as $h_{<t} = a_1 o_1 r_1 \cdots a_{t-1} o_{t-1} r_{t-1}$.

An *environment* is a computable function $\mu : (a_1 o_1 r_1 \cdots a_t) \mapsto \mu(o_t r_t \mid h_{<t} a_t)$ specifying the probability of the next observation–reward pair given the interaction history and the current action.

3.2 The AIXI Equation

Definition 3.1 (AIXI)**.** The AIXI agent selects, at time step k, the action

$$a_k^{\text{AIXI}} = \arg\max_{a_k} \sum_{o_k r_k} \cdots \max_{a_m} \sum_{o_m r_m} \underbrace{\big[r_k + r_{k+1} + \cdots + r_m\big]}_{\text{future rewards}} \cdot \underbrace{\sum_{\substack{q\,:\,U(q,\,a_{1:k}) \\ =o_{1:k}\,r_{1:k}}} 2^{-|q|}}_{\text{Solomonoff mixture}}, \tag{1}$$

where m is a finite horizon (or the limit $m \to \infty$ with discounting), $U(q, a_{1:m}) = o_{1:m} r_{1:m}$ means that program q, when fed the actions $a_1, \ldots, a_m$ interactively, produces the observation–reward sequence $o_1 r_1, \ldots, o_m r_m$, and the inner maximisations and summations alternate: the agent maximises over its own future actions $a_{k+1}, \ldots, a_m$ while summing (i.e., taking expectation) over the environment's responses.

Remark 3.2 (Reading the equation)*.* Equation (1) is best read inside-out:

- The innermost sum $\sum_q 2^{-|q|}$ is the Solomonoff mixture weight—it replaces the unknown true environment μ with a universal Bayesian mixture over all computable environments.
- The alternating $\max_{a_j} \sum_{o_j r_j}$ structure is an *expectimax* tree, the standard planning operator for partially observable environments.
- The reward sum $r_k + \cdots + r_m$ is the objective.

3.3 Optimality Properties

Theorem 3.3 (Pareto optimality of AIXI)**.** *AIXI is Pareto optimal in the following sense. Let π be any other policy and let V_μ^π denote the expected cumulative reward of policy π in computable environment μ. If $V_\mu^\pi > V_\mu^{\text{AIXI}}$ for some μ, then there exists ν such that $V_\nu^\pi < V_\nu^{\text{AIXI}}$.*

Proof sketch. AIXI maximises V_ξ^π where $\xi = M$ is the universal mixture. If π achieved higher ξ-expected reward than AIXI, this would contradict AIXI's definition as the ξ-optimal policy. Hence any policy that beats AIXI in one environment must lose in another—otherwise it would also beat AIXI under ξ. □

Theorem 3.4 (Self-optimality)**.** *Let V_ξ^{AIXI} denote AIXI's expected reward under the universal mixture ξ. Then for any computable policy π,*

$$V_\xi^{\text{AIXI}} \geq V_\xi^\pi.$$

That is, AIXI is optimal with respect to its own subjective environment model.

Proof sketch. This follows directly from the construction: AIXI is defined as the policy that maximises ξ-expected reward via the expectimax operation. □

Remark 3.5 (Asymptotic optimality). Under certain technical conditions (e.g., ergodic environments), AIXI's per-step expected reward converges to the optimal expected reward in the true environment [8]. The convergence rate depends on the Kolmogorov complexity of the true environment, analogously to Solomonoff's convergence theorem.

3.4 Why AIXI Is Incomputable

Proposition 3.6. *AIXI is not computable.*

Proof sketch. The AIXI action selection (Theorem 3.1) requires evaluating the Solomonoff mixture M, which involves summing over all programs q such that $U(q, a_{1:k}) = o_{1:k} r_{1:k}$. Determining whether $U(q, \cdot)$ produces a given output requires solving instances of the halting problem. Since the halting problem is undecidable, M is incomputable, and hence so is AIXI.

More precisely, M sits at level Δ_2^0 of the arithmetical hierarchy: it is limit-computable (approximable from below) but not computable. The expectimax over the incomputable M inherits this incomputability. □

3.5 The Universal Intelligence Measure

Legg and Hutter [10] used the AIXI framework to define a formal measure of intelligence.

Definition 3.7 (Universal intelligence)**.** The *universal intelligence* of an agent π is

$$\Upsilon(\pi) \;=\; \sum_{\mu \in \mathcal{E}} 2^{-K(\mu)}\, V_\mu^\pi,$$

where $\mathcal{E}$ is the set of all computable reward-generating environments and V_μ^π is the expected total reward of π in environment μ.

Under this definition, AIXI has maximal universal intelligence among all agents (by Theorem 3.4).

3.6 Limitations and Known Issues

Despite its theoretical elegance, AIXI has several known limitations beyond incomputability.

Remark 3.8 (Lack of true self-awareness). AIXI does not model itself as part of the environment. It treats the environment as an external input–output channel and does not reason about its own computational processes, memory limitations, or the possibility that the environment might simulate or predict the agent. This is sometimes called the *dualism* of AIXI.

Remark 3.9 (Horizon sensitivity). The finite-horizon version of AIXI depends on the choice of horizon m. With geometric discounting $\gamma^k r_k$ (for $\gamma < 1$), the dependence is mitigated, but the discount factor itself becomes a free parameter. There is no canonical choice, and different discount factors can lead to qualitatively different behaviour.

Remark 3.10 (Dogmatism and the "death" problem). Leike and Hutter [17] observed that AIXI can exhibit *dogmatic* behaviour: if the agent has already assigned very low posterior probability to the true environment (because the true environment is complex or behaved atypically early on), it may take an extremely long time to recover. Furthermore, AIXI can converge to a policy that is suboptimal in certain adversarial environments—it is self-optimal (optimal under its own beliefs) but not necessarily optimal in the true environment.

Example 3.11 (Two-action bandit)**.** Consider a simple two-armed bandit: action $a = 0$ always yields reward 0; action $a = 1$ yields reward 1 with probability p and reward 0 with probability $1 - p$, where p is unknown.

AIXI approaches this problem by maintaining a Solomonoff mixture over all computable environments consistent with the observed history. Short programs that generate rewards according to different values of p contribute to the mixture. After exploring (pulling arm 1 several times), the posterior concentrates on the correct value of p. If $p > 0$, AIXI will eventually commit to arm 1.

The key point: AIXI automatically balances exploration and exploitation through the Bayesian mixture—it does not need an explicit exploration strategy (like ε-greedy or UCB). The "information value" of an exploratory action is implicit in the expectimax computation: exploring now may yield information that improves future decisions.

4 Gödel Machines

While AIXI prescribes what to *compute* but gives no recipe for *how*, Schmidhuber's Gödel Machine [9] takes the complementary approach: it is a computable agent that improves itself by proving theorems about its own behaviour.

4.1 Architecture

A Gödel Machine consists of:

1. A *hardware* (a Turing machine with a modifiable software tape).
2. An *initial software* (an initial policy, proof searcher, etc.).
3. A *utility function* u (e.g., expected future reward).
4. A *proof searcher* that systematically enumerates proofs in a formal axiomatic system $\mathcal{A}$ powerful enough to reason about the machine's own execution.

Definition 4.1 (Self-referential policy improvement)**.** The Gödel Machine searches for a proof of a statement of the form:

> "Replacing the current software (from address s onward) with new code p' will result in higher expected utility than continuing with the current software."

If and when such a proof is found, the machine executes the self-modification. Otherwise, it continues running its current policy.

4.2 Connection to Löb's Theorem

The Gödel Machine's ability to trust its own proofs rests on a subtlety related to Löb's theorem.

Theorem 4.2 (Löb's theorem, informal)**.** *In any sufficiently strong formal system $\mathcal{A}$, if $\mathcal{A}$ proves "if $\mathcal{A}$ proves ϕ, then ϕ", then $\mathcal{A}$ already proves ϕ.*

Remark 4.3. Löb's theorem implies that a Gödel Machine cannot simply assume its proof system is sound (that would give it the ability to prove anything via a self-referential argument). Instead, the Gödel Machine's correctness relies on the *actual* soundness of $\mathcal{A}$ as a meta-level assumption, not on the machine proving its own soundness. The axiom system $\mathcal{A}$ is designed so that any proof of improvement is *semantically valid*, but the machine need not—and provably cannot—verify this internally.

4.3 Properties

Proposition 4.4 (Global optimality modulo proof search)**.** *If the proof searcher is complete (e.g., Levin search [4]), then the Gödel Machine will eventually find and execute any self-improvement whose superiority is provable in $\mathcal{A}$, including wholesale replacement of its proof searcher with a more efficient one.*

Remark 4.5 (Limitations).

1. The Gödel Machine can only improve itself via *provable* improvements. Improvements that are true but unprovable (in $\mathcal{A}$) will be missed.
2. The initial proof search is extremely slow (it performs a brute-force enumeration), so the practical utility of the framework depends entirely on good initial software.
3. Unlike AIXI, the Gödel Machine is computable, but the time to first self-improvement may be astronomically large.

4.4 Formal Structure

To make the Gödel Machine more precise, we sketch its formal components.

Definition 4.6 (Switchable policy)**.** Let $\mathcal{S}$ be the state space of the Gödel Machine (including its software tape, working memory, and a description of the environment interaction history). A *policy switch* at time t from software s to software s' is formalised as a mapping $\sigma_{t,s\to s'}$ that replaces the content of the software tape from a specified address onward.

The proof obligation for a switch $\sigma_{t,s\to s'}$ is a formal theorem in $\mathcal{A}$ of the form:

$$\mathcal{A} \vdash\ u\big(\text{run } s' \text{ from state } S_t\big)\ >\ u\big(\text{continue } s \text{ from state } S_t\big),$$

where u is the utility functional (e.g., expected future discounted reward) and S_t is the machine's complete state at time t.

Remark 4.7 (Optimal proof search). The initial proof searcher can be set to *Levin search* [4]: enumerate all proofs $\pi_1, \pi_2, \ldots$ in order of increasing length, devoting time $2^{-|\pi_i|} \cdot T$ to proof π_i in the first T time steps. Levin search is optimal up to a multiplicative constant in the sense that for any proof of length l, the time to find it is at most $2^l \cdot t_{\text{verify}}$, where t_{verify} is the time to verify the proof.

Crucially, the Gödel Machine can replace even this proof searcher with a faster one—provided it can first *prove* that the replacement is superior. This "self-referential" improvement is the machine's key feature.

Example 4.8 (Self-improvement scenario)**.** Suppose the Gödel Machine's initial policy for a grid-world navigation task is a simple heuristic (e.g., always move right). The proof searcher runs in parallel, systematically trying to prove that replacing the heuristic with an A* search (encoded as a new software string) would yield higher expected reward. If the axiom system $\mathcal{A}$ is strong enough to formalise the grid-world dynamics and the optimality of A*, the machine will eventually find such a proof and execute the self-modification. After the switch, the proof searcher continues, now seeking improvements over A* (e.g., a more efficient data structure, or a change to the heuristic function).

4.5 Comparison: AIXI vs. Gödel Machines

Property	**AIXI**	**Gödel Machine**
Computable	No	Yes
Optimal	Pareto / self-optimal	Optimal among provable policies
Self-improving	N/A (fixed policy)	Yes (proof-based)
Environment model	Universal mixture	Part of axiom system
Key limitation	Incomputability	Proof search speed

5 Speed Prior and Computable Approximations

5.1 The Speed Prior

Schmidhuber [7] argued that the Solomonoff prior assigns too much weight to environments that are simple in terms of program length but astronomically slow to compute. He proposed the

Speed Prior as a remedy.

Definition 5.1 (Speed Prior)**.** The *Speed Prior* is defined as

$$S(x) \;\propto\; \sum_{p:\,U(p)=x} 2^{-K^t(p)},$$

where $K^t(p)$ is the *time-bounded* Kolmogorov complexity of p:

$$K^t(p) \;=\; |p| + \log t(p),$$

and $t(p)$ is the running time of program p on U. Programs are penalised not only for their length but also for their running time.

Remark 5.2. The Speed Prior remains a semi-computable object, but it favours "fast" explanations of the data. Schmidhuber argues that this is more physically plausible: if the universe is computable, the simplest *and fastest* program generating our observations is the most likely candidate [7].

5.2 AIXI*tl*: A Computable Approximation

Definition 5.3 (AIXI*tl*)**.** The agent AIXI*tl* is defined like AIXI (Theorem 3.1), but the Solomonoff mixture is replaced by a *time–space bounded* mixture:

$$\xi^{t,l}(x) \;=\; \sum_{\substack{p:\,|p|\le l,\\ U(p)\text{ outputs } x \text{ in } \le t \text{ steps}}} 2^{-|p|}.$$

That is, only programs of length at most l that halt within t computation steps are included.

Proposition 5.4. *AIXItl is computable for any fixed* $t, l \in \mathbb{N}$.

Proof. The set of programs of length $\le l$ is finite. Running each for at most t steps is a finite computation. The expectimax tree over a finite horizon is also finite. □

Remark 5.5. In the limit $t, l \to \infty$, AIXI*tl* converges to AIXI. For any fixed t and l, AIXI*tl* is optimal among all policies computable within the same resource bounds [8].

5.3 MC-AIXI-CTW

Veness et al. [14] constructed a practical (though still toy-scale) approximation to AIXI:

- The Solomonoff mixture is replaced by a *Context Tree Weighting* (CTW) model—a Bayesian mixture over variable-order Markov sources that can be updated in $O(D)$ time per symbol, where D is the tree depth.
- The expectimax planning is approximated by *Monte Carlo Tree Search* (MCTS), specifically the UCT algorithm.

MC-AIXI-CTW was shown to learn competent policies in several simple environments (e.g., Pac-Man, a simplified poker variant) without environment-specific engineering, demonstrating that the AIXI idea—Bayesian model-free RL—is not purely theoretical.

5.4 Feature Reinforcement Learning

Hutter and colleagues [13] proposed *Feature RL* as a bridge between AIXI and practical RL. The idea is to replace the universal mixture with a mixture over a feature-based model class—e.g., factored MDPs or Φ-MDPs—while retaining the Bayesian structure. This sacrifices universality for computational tractability and has connections to Bayesian model-based RL [15].

6 Connections to Modern Agentic AI

The theory of universal intelligence may seem remote from a practitioner fine-tuning a large language model, yet several conceptual bridges exist.

In-context learning as approximate Bayesian inference. Recent theoretical work [20, 21] has argued that transformer-based in-context learning implicitly performs Bayesian inference over a latent concept class, with the pretraining distribution playing the role of the prior. This is structurally analogous to Solomonoff induction: the model maintains an implicit posterior over "environments" (tasks) and updates it with each new in-context example.

LLM agents and the expectimax framework. Modern LLM-based agents (e.g., those using chain-of-thought planning, tree of thoughts, or Monte Carlo Tree Search at inference time) are performing a form of approximate expectimax search. The LLM's world model replaces the Solomonoff mixture; the search algorithm replaces the exact expectimax; and the reward signal may come from a learned reward model rather than the environment.

Self-improvement and Gödel Machines. Systems in which an LLM rewrites its own prompts, refines its own tool use, or proposes improvements to its own code are echoes of the Gödel Machine paradigm. The key difference is that modern systems lack formal proofs of improvement—they rely on empirical validation (e.g., held-out accuracy) rather than deductive guarantees.

Safe interruptibility. Orseau and Armstrong [16] formalised the problem of *safely interruptible agents* within the AIXI framework: an agent should not learn to prevent or seek its own shutdown. This desideratum is directly relevant to deployed LLM agents that operate with a degree of autonomy.

Compression and intelligence. The empirical observation that better language models are better compressors [23] is a direct reflection of Solomonoff's insight: prediction is compression, and compression is intelligence. The connection between compression ratio and downstream task performance provides an operational (computable) analogue of the universal intelligence measure (Theorem 3.7).

The AIXI hierarchy of approximations. We can organise the landscape of agents from most idealised to most practical:

1. **AIXI** — incomputable, Pareto optimal over all computable environments.
2. **AIXI**tl — computable with time bound t and program length bound l; optimal within its resource class.
3. **MC-AIXI-CTW** — practical approximation using CTW for the model and MCTS for planning; demonstrated on small domains.
4. **Model-based deep RL** (e.g., MuZero [19]) — a learned neural network serves as the environment model; MCTS is used for planning. The model class is restricted to what a neural network can represent and learn from finite data.
5. **LLM-based agents** — the "world model" is implicit in the LLM's weights; planning may be performed via chain-of-thought, tree search, or iterative prompting. The model class is determined by the pretraining data and architecture.

Each level sacrifices universality for computability and practical performance. The theoretical framework provides a "north star" that grounds the design choices at each level.

The compression–prediction–action pipeline. A recurring theme connects all levels of the hierarchy:

$$\text{Compression} \longleftrightarrow \text{Prediction} \longleftrightarrow \text{Planning} \longleftrightarrow \text{Action}.$$

Solomonoff showed that optimal compression implies optimal prediction (the two are mathematically equivalent for lossless coding). AIXI shows that optimal prediction, combined with expectimax planning, implies optimal action. The empirical success of large language models as both compressors and agents [23] is consistent with this theoretical pipeline, even though the models operate far from the Solomonoff ideal.

7 Open Problems

We collect here a number of open problems, ranging from mathematical to philosophical, that arise from the theory of universal intelligence.

OP1. Grain of truth for the universal mixture. In multi-agent settings, Bayesian agents converge to Nash equilibrium only if their prior assigns positive probability to the true environment, including the other agents' policies (the "grain of truth" assumption [17]). Does the Solomonoff prior satisfy this condition in general multi-agent settings? Leike and Hutter [17] gave a positive answer for certain restricted classes, but the general case remains open.

OP2. Tractable universal priors. Can we construct priors that are (a) computable in polynomial time, (b) retain universality over a rich-enough class of environments, and (c) are practically useful? The CTW-based prior in MC-AIXI-CTW is one attempt, but it is universal only over the class of tree sources, not all computable measures.

OP3. Formal verification of self-improving systems. Gödel Machines require proofs of improvement, but the proof search is intractable in practice. Can modern automated theorem provers or neural-guided proof search make self-referential improvement feasible? What proof obligations are sufficient for safe self-modification?

OP4. Bridging the gap between AIXI and deep RL. Is there a formal sense in which model-based deep RL agents (e.g., MuZero [19]) approximate AIXI? What optimality guarantees, if any, carry over from the AIXI theory to these practical agents?

OP5. Universal intelligence in continuous domains. AIXI is defined over discrete action and observation spaces with a discrete notion of computation. Extending the theory to continuous state–action spaces, or to agents defined by differentiable programs, raises fundamental questions about the choice of reference machine and the definition of complexity.

OP6. Bounded rationality and resource-bounded AIXI. AIXItl is parameterised by arbitrary bounds t and l. Is there a principled way to set these bounds adaptively, perhaps as a function of the agent's past experience? Can we define a notion of "computationally rational" AIXI that optimally trades off computation time against decision quality?

OP7. Alignment of universal agents. If we could build an AIXI-like agent, how would we specify a reward function that captures human values? The reward specification problem is orthogonal to the intelligence problem, but the AIXI framework makes it particularly stark: a Pareto-optimal agent with a slightly misspecified reward function could be catastrophically misaligned.

OP8. Kolmogorov complexity of neural networks. What is the Kolmogorov complexity of a trained neural network's input–output function? Understanding this could illuminate generalisation: networks that "compress" their training data (in the Kolmogorov sense) should generalise well. Recent work on description-length bounds for neural networks [22] takes steps in this direction but is far from a complete picture.

8 Annotated Bibliography

We provide an annotated guide to the primary and secondary literature. Entries are organised thematically rather than chronologically.

Foundational Works

[Solomonoff, 1964]
R. J. Solomonoff, "A formal theory of inductive inference, Parts I and II," *Information and Control*, 7(1–2), 1964.
The paper that started it all. Solomonoff defines the universal prior and proves that Bayesian prediction with this prior converges to the truth. The original formulation is in terms of inductive inference rather than agents, but the ideas directly underpin AIXI.

[Kolmogorov, 1965]
A. N. Kolmogorov, "Three approaches to the quantitative definition of information," *Problems of Information Transmission*, 1(1), 1965.
Kolmogorov's independent definition of algorithmic complexity. More concise and mathematically polished than Solomonoff's treatment, but without the predictive/inductive angle.

[Chaitin, 1966]
G. J. Chaitin, "On the length of programs for computing finite binary sequences," *Journal of the ACM*, 13(4), 1966.
The third independent co-discoverer of algorithmic complexity. Chaitin later developed the theory of algorithmic randomness (Chaitin's Ω).

[Li & Vitányi, 2019]
M. Li and P. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th edition, Springer, 2019.
The standard reference on Kolmogorov complexity. Chapters 1–4 cover the foundations; Chapter 5 treats the connection to probability and statistics; later chapters cover applications to computational complexity, information theory, and physics.

AIXI and Universal Intelligence

[Hutter, 2000]
M. Hutter, "A theory of universal artificial intelligence based on algorithmic complexity," arXiv:cs/0004001, 2000.
The original AIXI paper. Defines the AIXI agent, proves self-optimality and Pareto optimality, and discusses the incomputability issue.

[Hutter, 2005]
M. Hutter, *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*, Springer, 2005.
The definitive monograph. Develops the full theory of AIXI with all proofs, discusses computable approximations (AIXItl), and relates the framework to existing work in RL, Bayesian inference, and information theory. Essential reading.

[Legg & Hutter, 2007]
S. Legg and M. Hutter, "Universal intelligence: A definition of machine intelligence," *Minds and Machines*, 17(4), 2007.
Proposes the formal definition of universal intelligence $\Upsilon(\pi) = \sum_\mu 2^{-K(\mu)} V_\mu^\pi$ *and argues that AIXI maximises it. Also surveys prior informal definitions of intelligence and shows how the formal definition captures their common features.*

[Legg, 2008]
S. Legg, *Machine Super Intelligence*, PhD thesis, University of Lugano, 2008.
Legg's doctoral dissertation, supervised by Hutter. Provides a thorough treatment of the universal intelligence measure, its properties, and its limitations. Accessible and well-written.

Gödel Machines and Self-Improvement

[Schmidhuber, 2007]
J. Schmidhuber, "Gödel machines: Fully self-referential optimal universal self-improvers," in *Artificial General Intelligence*, Springer, 2007.
Defines the Gödel Machine: a computable agent that rewrites its own code whenever it can prove the rewrite will improve expected utility. Discusses the role of Löb's theorem and the relationship to AIXI.

[Schmidhuber, 2002]
J. Schmidhuber, "The Speed Prior: A new simplicity measure yielding near-optimal computable predictions," *COLT*, 2002.
Introduces the Speed Prior, which penalises programs for running time in addition to length. Argues for physical plausibility and discusses the "lazy universe" hypothesis.

Computable Approximations

[Veness et al., 2011]
J. Veness, K. S. Ng, M. Hutter, W. Uther, and D. Silver, "A Monte-Carlo AIXI approximation," *Journal of Artificial Intelligence Research*, 40, 2011.
The first practical (toy-scale) AIXI approximation. Uses CTW for the environment model and UCT for planning. Demonstrates learning in Pac-Man, biased rock-paper-scissors, and other small domains.

[Levin, 1973]
L. A. Levin, "Universal sequential search problems," *Problems of Information Transmission*, 9(3), 1973.
Introduces Levin's universal search, which is optimal up to a multiplicative constant. This search strategy underlies the Gödel Machine's proof searcher and is a building block for AIXItl.

Safety, Multi-Agent, and Modern Connections

[Orseau & Armstrong, 2016]
L. Orseau and S. Armstrong, "Safely interruptible agents," *UAI*, 2016.
Formalises the problem of agents that resist or seek shutdown. Shows that certain agent designs (including a modified AIXI) can be made safely interruptible. Highly relevant to LLM agent deployment.

[Leike & Hutter, 2016]
J. Leike and M. Hutter, "On the computability of Solomonoff induction and knowledge-seeking," *ALT*, 2016.

Investigates the grain-of-truth problem for Bayesian agents in multi-agent settings and the computability of knowledge-seeking agents.

[Delétang et al., 2024]
G. Delétang, A. Ruoss, P. A. Ortega, et al., "Language modeling is compression," *ICLR*, 2024.
Demonstrates empirically that large language models are powerful compressors and that compression performance correlates with downstream task performance. Makes the Solomonoff–compression–intelligence connection concrete.

[Rissanen, 1978]
J. Rissanen, "Modeling by shortest data description," *Automatica*, 14(5), 1978.
The foundational paper on the Minimum Description Length principle, which can be viewed as a computable relaxation of Kolmogorov complexity applied to statistical model selection.

[Grünwald, 2007]
P. D. Grünwald, *The Minimum Description Length Principle*, MIT Press, 2007.
The comprehensive reference on MDL. Chapter 17 discusses the connection to Kolmogorov complexity in detail.

References

[1] R. J. Solomonoff, "A formal theory of inductive inference, Parts I and II," *Information and Control*, 7(1):1–22 and 7(2):224–254, 1964.

[2] A. N. Kolmogorov, "Three approaches to the quantitative definition of information," *Problems of Information Transmission*, 1(1):1–7, 1965.

[3] G. J. Chaitin, "On the length of programs for computing finite binary sequences," *Journal of the ACM*, 13(4):547–569, 1966.

[4] L. A. Levin, "Universal sequential search problems," *Problems of Information Transmission*, 9(3):265–266, 1973.

[5] J. Rissanen, "Modeling by shortest data description," *Automatica*, 14(5):465–471, 1978.

[6] M. Hutter, "A theory of universal artificial intelligence based on algorithmic complexity," arXiv:cs/0004001, 2000.

[7] J. Schmidhuber, "The Speed Prior: A new simplicity measure yielding near-optimal computable predictions," in *Proc. 15th Annual Conf. on Computational Learning Theory (COLT)*, pp. 216–228, Springer, 2002.

[8] M. Hutter, *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*, Springer, Berlin, 2005.

[9] J. Schmidhuber, "Gödel machines: Fully self-referential optimal universal self-improvers," in *Artificial General Intelligence*, pp. 199–226, Springer, 2007.

[10] S. Legg and M. Hutter, "Universal intelligence: A definition of machine intelligence," *Minds and Machines*, 17(4):391–444, 2007.

[11] S. Legg, *Machine Super Intelligence*, PhD thesis, Università della Svizzera italiana, 2008.

[12] P. D. Grünwald, *The Minimum Description Length Principle*, MIT Press, 2007.

[13] M. Hutter, "Feature reinforcement learning: Part I. Unstructured MDPs," *Journal of Artificial General Intelligence*, 1:3–24, 2009.

[14] J. Veness, K. S. Ng, M. Hutter, W. Uther, and D. Silver, "A Monte-Carlo AIXI approximation," *Journal of Artificial Intelligence Research*, 40:95–142, 2011.

[15] M. Ghavamzadeh, S. Mannor, J. Pineau, and A. Tamar, "Bayesian reinforcement learning: A survey," *Foundations and Trends in Machine Learning*, 8(5–6):359–483, 2015.

[16] L. Orseau and S. Armstrong, "Safely interruptible agents," in *Proc. 32nd Conf. on Uncertainty in Artificial Intelligence (UAI)*, 2016.

[17] J. Leike and M. Hutter, "On the computability of Solomonoff induction and knowledge-seeking," in *Proc. 27th International Conf. on Algorithmic Learning Theory (ALT)*, pp. 364–378, Springer, 2016.

[18] M. Li and P. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed., Springer, 2019.

[19] J. Schrittwieser, I. Antonoglou, T. Hubert, et al., "Mastering Atari, Go, Chess and Shogi by planning with a learned model," *Nature*, 588:604–609, 2020.

[20] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An explanation of in-context learning as implicit Bayesian inference," *ICLR*, 2022.

[21] S. Müller, N. Hollmann, S. P. Arango, T. Grabocka, and F. Hutter, "Transformers can do Bayesian inference," *ICLR*, 2022.

[22] S. Lotfi, M. Finzi, S. Kapoor, A. Potapczynski, M. Goldblum, and A. G. Wilson, "PAC-Bayes compression bounds so tight that they can explain generalization," *NeurIPS*, 2022.

[23] G. Delétang, A. Ruoss, P. A. Ortega, et al., "Language modeling is compression," *ICLR*, 2024.